#3.1-Write a Python code for the integrand

The trapezium rule approximates a function by splitting it into multiple trapezia and dividing by h(width). A function trap0 was used to perform the integration It divides the integration range into n small intervals, evaluates the function at each point, and sums the areas of the trapezia formed under the curve.
This function can be used for any integrand by simply passing it as an argument, is needed for later parts of the code.

#3.2-Value of integrand

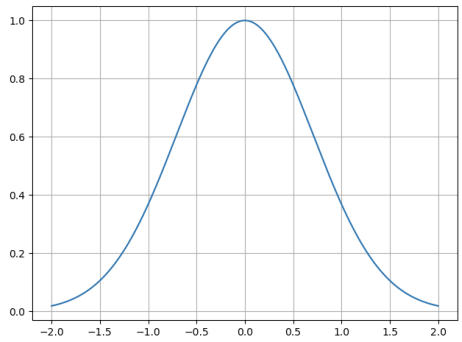
I inputted the integral using the limits of x=0,1 and used n=10. The more n's the more accurate it is. My value of the integrand was - `Value of the integrand is, 0.0012645110928411146-`

#3.3-Guess of how many trapeziums

For this one I varied n from my initial value of n. From the values 10,11,12,13,14,15,20 and then did an extreme of 200 and saw at which value the integrand didn't change up to 6 sigfigs and I found that this happen at n=14

#3.4-Try different ranges for a different integral

This integrand already has a known value which is around 1.772454. The limits of infinity and negative infinity were way too large to use on python so I used values of a range from –1,1 - -5,5 to find the one that was closest to 1.77 and found that it was –2,2. It gave me a value of 1.76226.

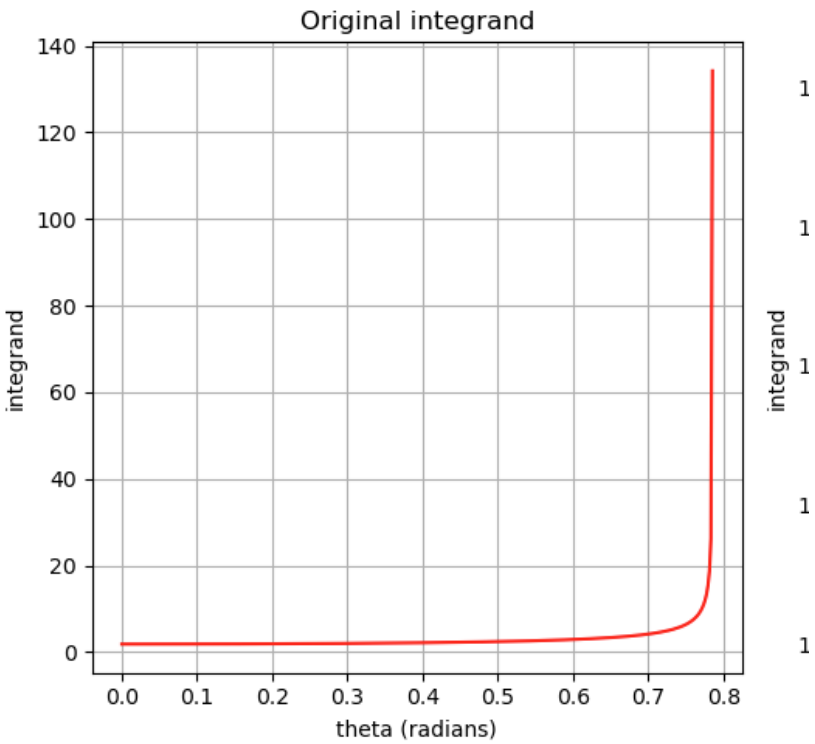


demonstrates the convergence behaviour of the trapezium rule — how its accuracy improves as we increase the number of slices. It confirms that the method works and approaches the true value smoothly for well-behaved functions.

#3.5-Plot the above Integrand

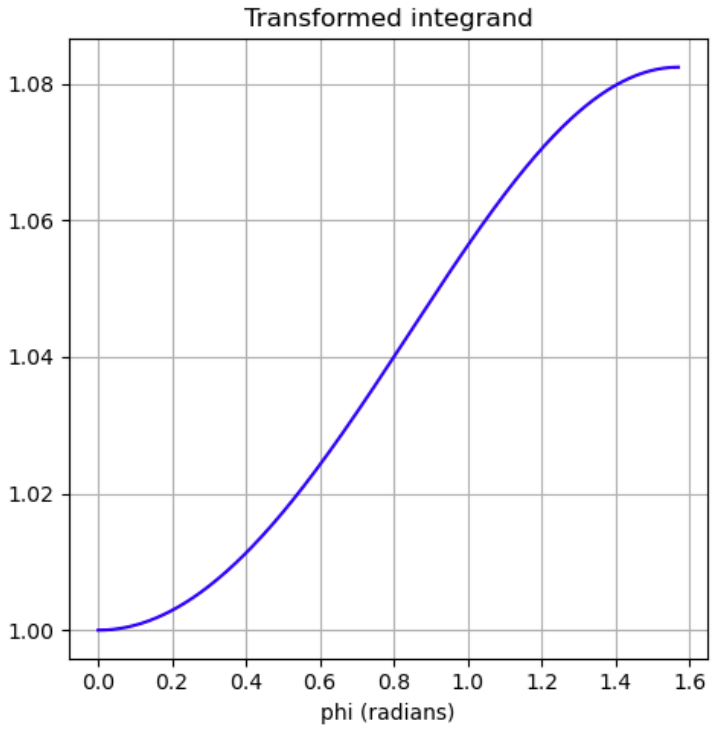
$$T = T_0 \frac{\sqrt{2}}{\pi} \int_0^{\alpha} \frac{d\theta}{(\cos\theta - \cos\alpha)^{1/2}}$$

This was the pendulum swing we were trying to plot. It is supposed to fail at alpha=45 degrees. Any numerical evaluation should fail as the integrand goes to infinity at the upper end point. If we change the variable it will woke. I wrote a function called integrand to show the original integrand. Then plotted with theta values between 0 and alpha then plotted the graph



The graph showed that the integrand increases sharply near θ = α, tending towards infinity. This confirms why the trapezium rule struggles for large angles: most of the area under the curve is concentrated in this steep region, making accurate approximation difficult.

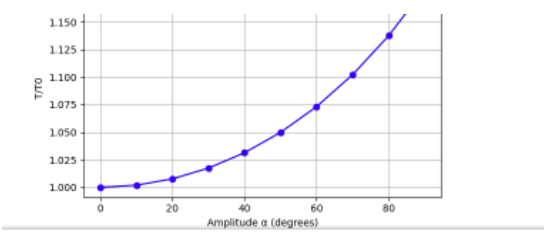
I did the recommended variable substitution and generated that graph for **BONUS WORK.**



The transformed integrand remains smooth and well behaved across the whole value for radians. This demonstrates how the substitution removes the singularity and allows accurate integration even for large amplitudes like 90°

#3.6-Ratio for T/T0

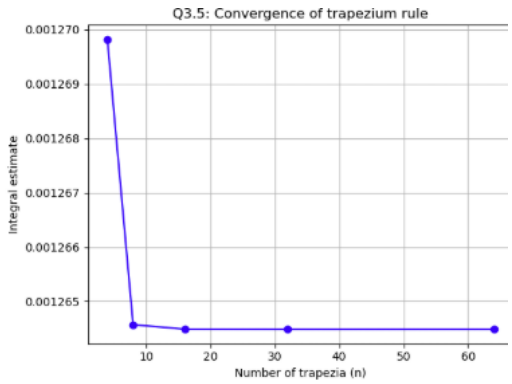
To achieve this, two numerical integration functions were implemented: trap0, which performs the basic trapezium rule integration, and trap1, an adaptive version that automatically refines the number of trapezia until the result meets a set accuracy threshold.




```
Amplitude = 0.0 degrees,  T/T0 = 1.0
Amplitude = 10.0 degrees,  T/T0 = 1.001907
Amplitude = 20.0 degrees,  T/T0 = 1.007669
Amplitude = 30.0 degrees,  T/T0 = 1.017409
Amplitude = 40.0 degrees,  T/T0 = 1.031341
Amplitude = 50.0 degrees,  T/T0 = 1.049783
Amplitude = 60.0 degrees,  T/T0 = 1.073182
Amplitude = 70.0 degrees,  T/T0 = 1.102145
Amplitude = 80.0 degrees,  T/T0 = 1.137493
Amplitude = 90.0 degrees,  T/T0 = 1.180341
```

The code computed amplitudes between 0° **and** 90°, and results were tabulated and plotted. The graph shows that T/T0 increases gradually with amplitude, confirming that a real pendulum takes longer to complete a swing at larger angles due to the

nonlinearity of the restoring force. For small amplitudes the ratio mains close to 1, validating the small-angle approximation. T/T0 at 90 was 1.18 which is like theoretical expectations



The adaptive trapezium rule was also tested for convergence. As shown in the convergence plot, the estimated integral stabilized at **n = 64**, giving a final value of approximately 0.0012645. This indicates that 64 trapezia were sufficient to reach the target precision of 6 significant figures showing that the integrand is smooth and the numerical method reliable.

#3.7-Accurate value at 90 degrees

In this section, I had to calculate an accurate value of the period ratio T/T_0 for a pendulum released from an amplitude of 90 degrees. Using the adaptive trapezium rule function T_ratio with a delta = 1×10^{-7}, the integration refined itself automatically until the result stabilized. The computed value of T/T_0 was approximately 1.18. This shows that at large amplitudes, the pendulum's period is about 18% longer than predicted by the small-angle approximation, confirming that nonlinear effects become significant at higher swing angles. The result demonstrates both the reliability of the adaptive integration method and the physical limitation of the small-angle model. Still aligned with our model

#Bonus work: QUAD

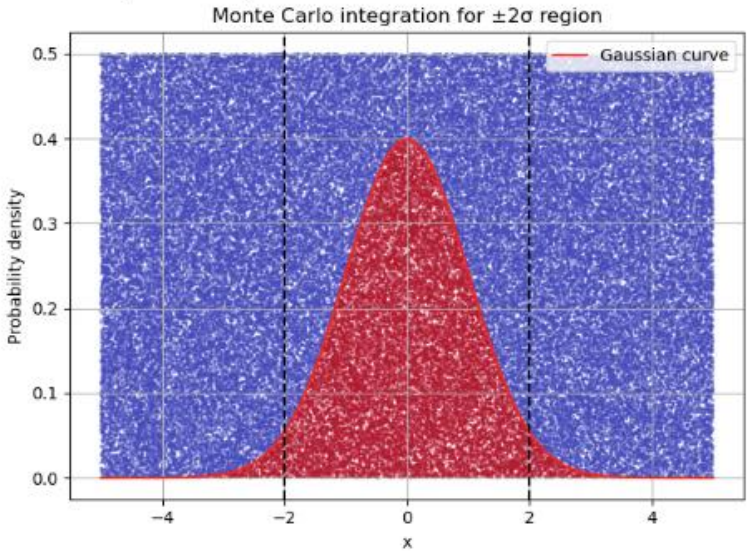
As a quick accuracy check, the adaptive trapezium method (trap1) was compared with Python's built-in SciPy quad function for the test integralof sinx(x) with limts 0-pu which equals 2 exactly. Both methods gave nearly identical results, with a relative error of about 10^-6. This shows that the adaptive trapezium rule works very well for smooth functions, with any small differences caused only by rounding and numerical precision.

#3.8-Monte Carlo

I used a Monte Carlo method to estimate the probability that a normally distributed variable lies within ±2σ of the mean. Random points were generated uniformly over a rectangular area (x from −5 to 5 and y from 0 to 0.5), and the fraction that landed below the Gaussian curve was used to approximate the area

under it. The estimated total area was 1.0022, which is very close to the expected value of 1, confirming that the sampling covered the full Gaussian. The estimated probability within ±2σ was 0.382, showing that while the method worked it under-sampled the narrower region around the mean. The plot shows the Gaussian curve in red with random points scattered across the domain, visually illustrating how random sampling approximates integration.

```
Total area =  1.00215
Estimated total area under Gaussian = 1.0022
Estimated probability within ±2σ = 0.382
```



DISCLAIMER:USED MORE THAN 5% AI IN CODE